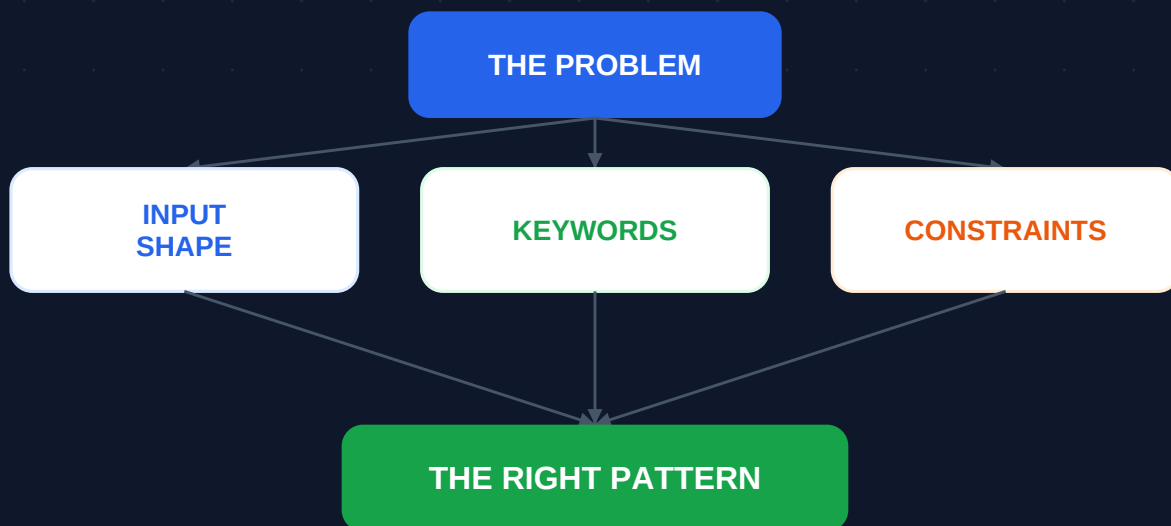


TIER-1 & GULF INTERVIEW PREP

DSA Pattern Selection Guide

How to read ANY problem's input, keywords & constraints and know the correct pattern before you write a line of code



Two Pointers

Sliding Window

HashMap

Prefix Sum

Binary Search

BFS/DFS

Backtracking

DP

Greedy

Heap

Union-Find

Monotonic Stack

Compiled for Wasim — Senior Software Engineer, targeting Gulf market SSE / Tech Lead roles

Visual framework inspired by Codewithnishchal's Array/String tricks flowchart

The 3-Signal Method

Every DSA problem gives away its pattern before you write any code. Train yourself to read three signals in this order — input shape, keywords, constraints — and cross-check them against each other. When two signals agree, you have your pattern.

1

INPUT SHAPE

What data structure am I given?
Array, string, linked list, tree, graph,
matrix, or a plain number?

→ narrows the whole pattern family

2

KEYWORDS

What is the problem asking for?
"Longest", "all pairs", "kth largest",
"all combinations", "minimum
window"?

→ narrows the exact technique

3

CONSTRAINTS

How large is N? What's the time
limit? Constraints silently tell you the
expected time complexity.

→ confirms or rules out a pattern

Worked Example — reading a problem in 10 seconds

"Given an array of integers and a target k, find the length of the longest subarray whose sum equals k. $1 \leq N \leq 10^5$."

INPUT

Array of integers

Array/String family — rules out tree/graph
patterns

KEYWORD

"longest subarray" + "sum equals k"

Contiguous range + running sum →
screams Prefix Sum (with HashMap)

CONSTRAINT

$N \leq 10^5$

Needs $O(N)$ or $O(N \log N)$ — rules out
 $O(N^2)$ brute force

All 3 signals agree → Pattern = Prefix Sum + HashMap | Target: $O(N)$ time, $O(N)$ space

When signals seem to conflict

- **Array looks sortable but problem needs original order preserved**

Trust the keyword. "Original order" / "indices" rules out sorting-based Two Pointers — use HashMap instead.

- **Constraint allows $O(N^2)$ but keyword screams Sliding Window**

Either works. In an interview, mention brute force in one line, then optimize — that sequence itself earns points.

- **Problem mentions a tree, but constraint is $N \leq 10^6$ with tight time limit**

It's likely a balanced/complete tree → expects $O(N)$ or $O(N \log N)$ traversal, not exponential recursion without memoization.

Master Pattern Map — Start Here

Step one, always: identify the input's data structure. This alone eliminates 70% of wrong patterns. Find your category below, then flip to the page number for the full decision flowchart.

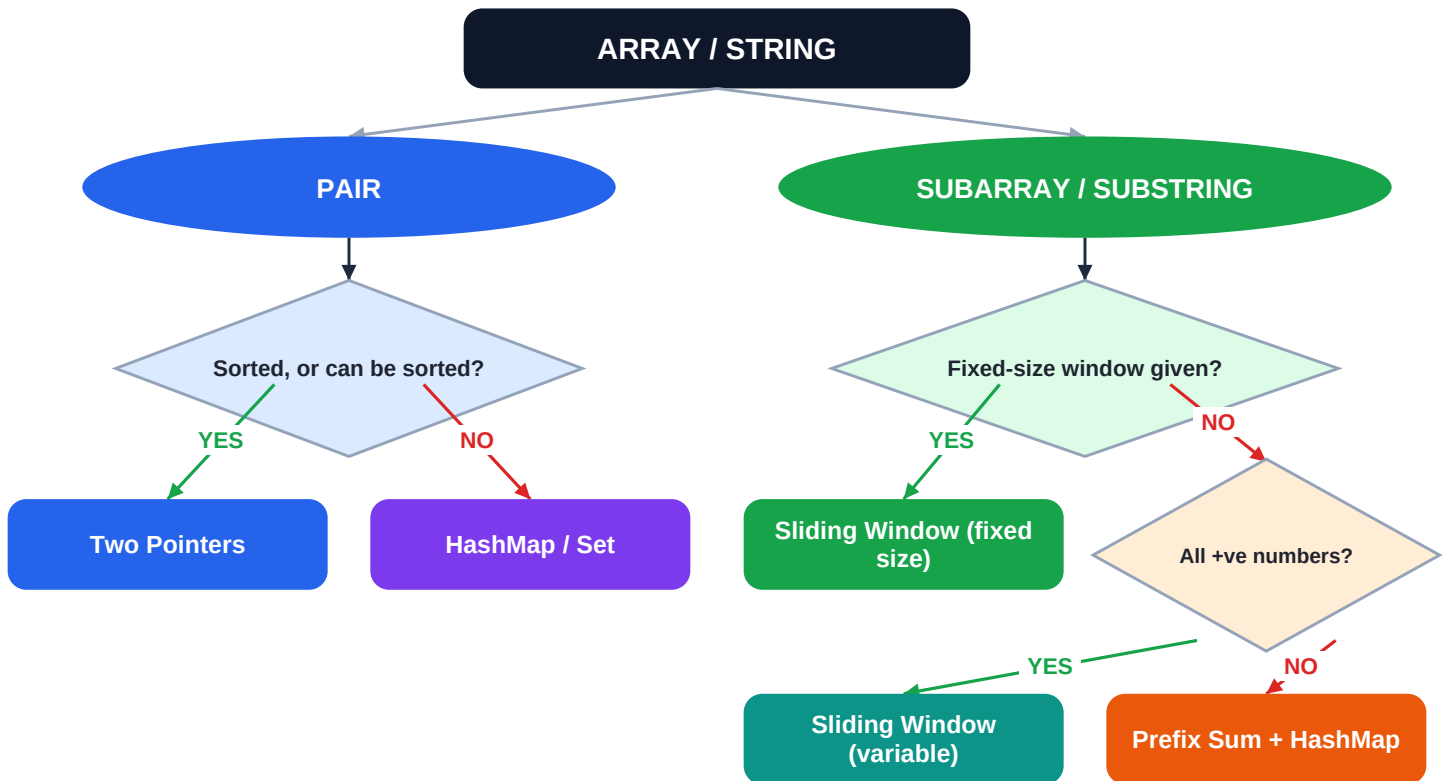


Quick gut-check

- Given indices / need to modify in place → Array family, not HashMap-only.
- “Connected”, “islands”, “neighbors”, “paths between nodes” → Graph, even if it's drawn as a grid.
- Recursion feels natural and input is hierarchical → Tree family.
- You keep undoing the last choice mentally → Backtracking, regardless of input shape (see p.8).

Pattern Flowchart: Pairs & Subarrays

The two most common array/string question families: forming PAIRS and analyzing SUBARRAYS or SUBSTRINGS (contiguous ranges). Follow the arrows to your pattern.



Pattern Notes — signal, complexity & examples

Two Pointers

Two indices move toward/away from each other over sorted data.

$O(N)$ time · $O(1)$ space

Ex: Two Sum II, 3Sum, Container With Most Water

HashMap / Set

Store seen values for $O(1)$ lookup when order/sort isn't usable.

$O(N)$ time · $O(N)$ space

Ex: Two Sum, Pair with given difference

Sliding Window (fixed)

Window size k slides across the array; add new, drop old.

$O(N)$ time · $O(1)$ space

Ex: Max sum subarray of size k

Sliding Window (variable)

Expand right pointer; shrink left while a condition breaks.

$O(N)$ time · $O(1)$ space

Ex: Longest substring without repeating chars

Prefix Sum + HashMap

Negatives break sliding window — track running sums instead.

$O(N)$ time · $O(N)$ space

Ex: Subarray Sum Equals K

Common mistakes interviewers watch for

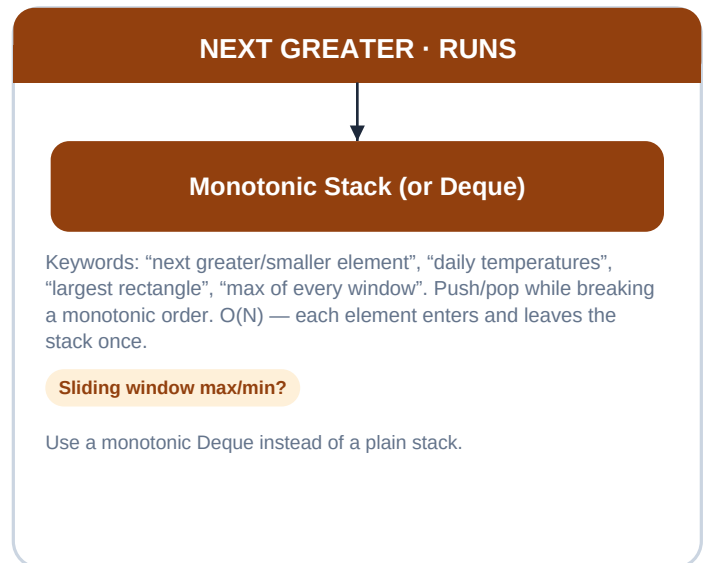
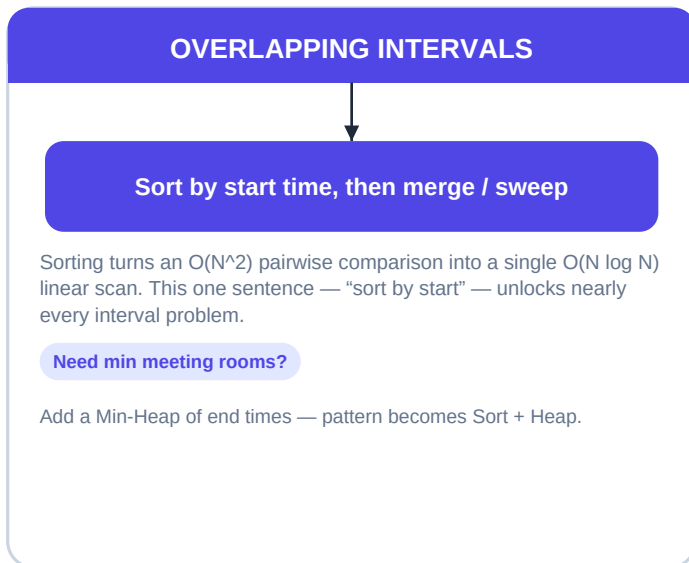
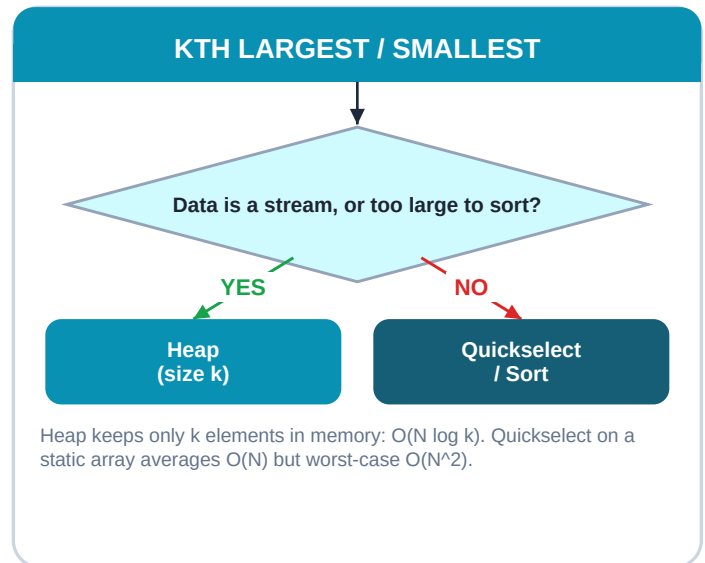
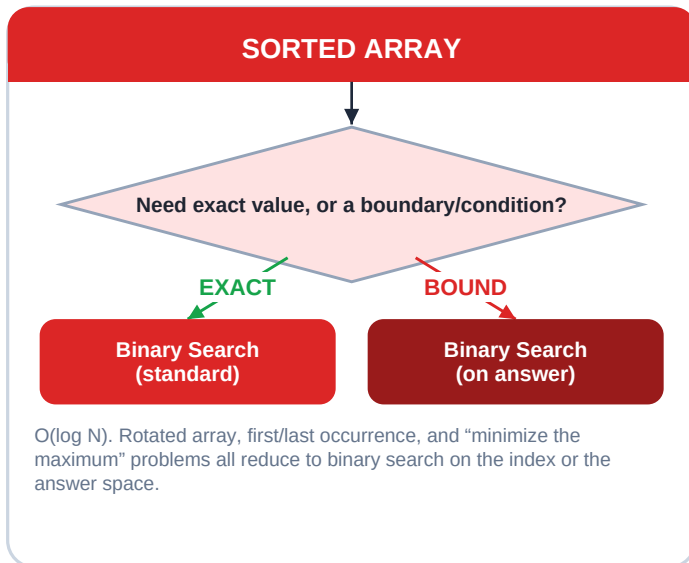
Jumping to Sliding Window when the array has negative numbers — the window sum stops being monotonic, so shrinking logic silently breaks.

Using Two Pointers on unsorted data without asking “can I sort this?” first — costs correctness, not just elegance.

Reaching for HashMap by default — it works but costs $O(N)$ space; say out loud why Two Pointers ($O(1)$ space) isn't viable first.

Sorted Arrays, Kth Element & Runs

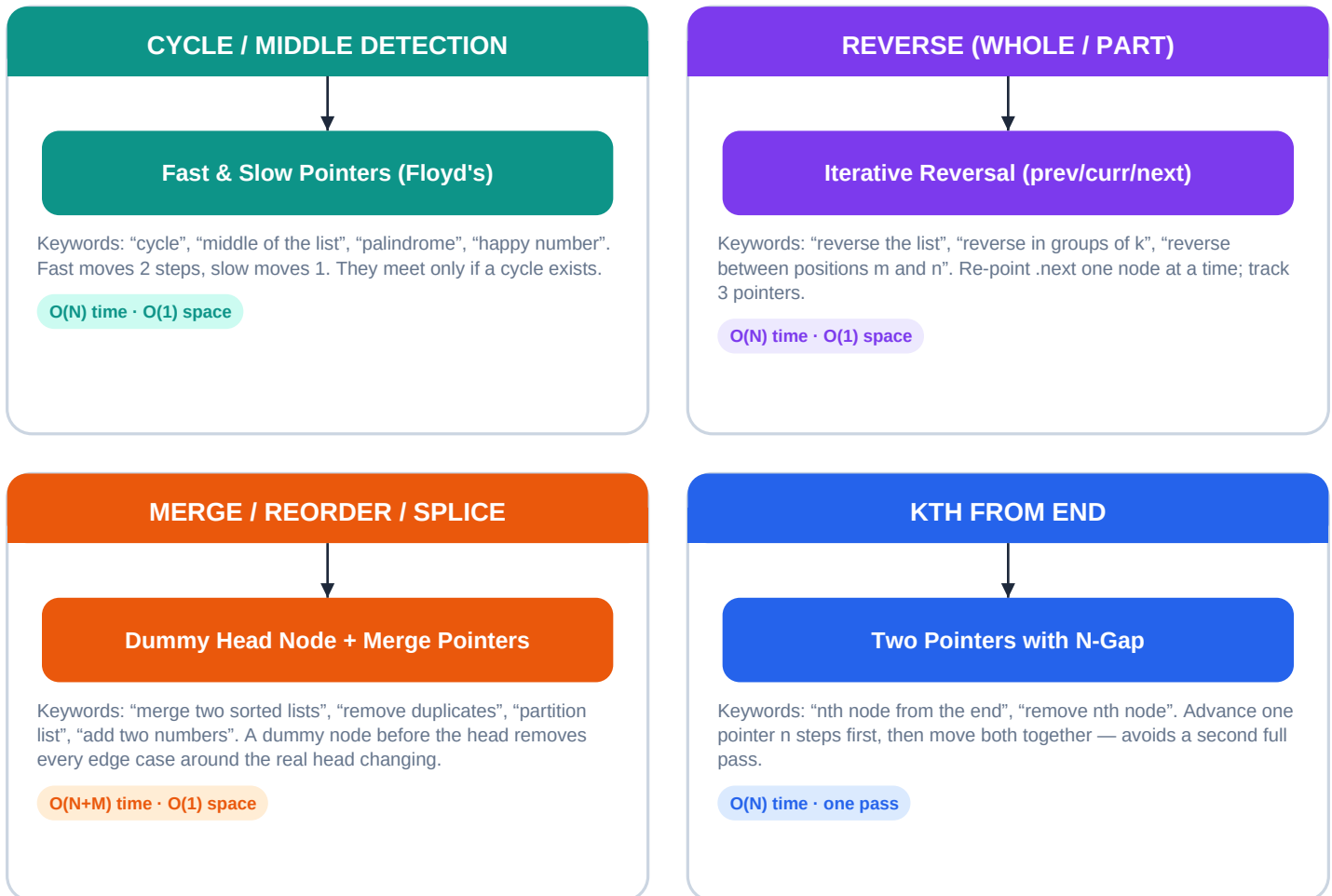
Four more signals that show up constantly in Array/String interviews. Each behaves independently of the Pair/Subarray split on the previous page — check these first if they apply.



Say the signal before the pattern: “since it’s sorted, I can binary search” beats jumping straight to code — every single time, in every interview.

Pattern Flowchart: Linked Lists

Linked lists have only four real patterns. If you can't modify values in place or need an array, you've probably misread the problem — pointers are almost always the answer.



The one habit that prevents 90% of linked-list bugs

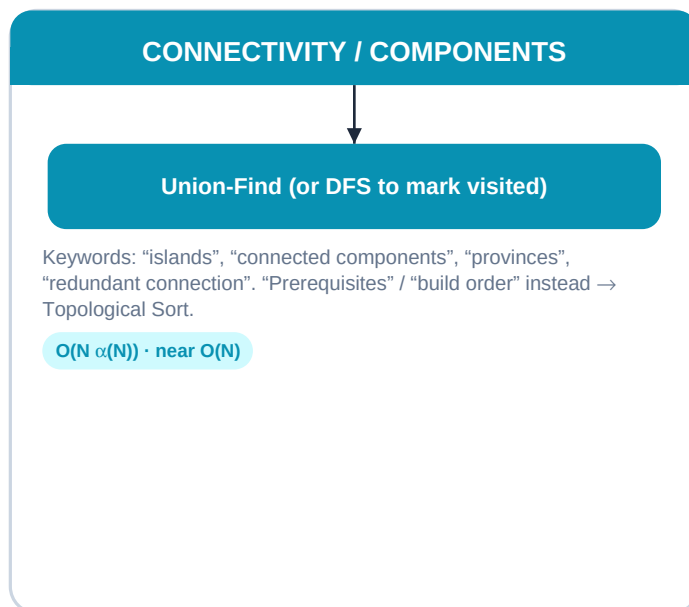
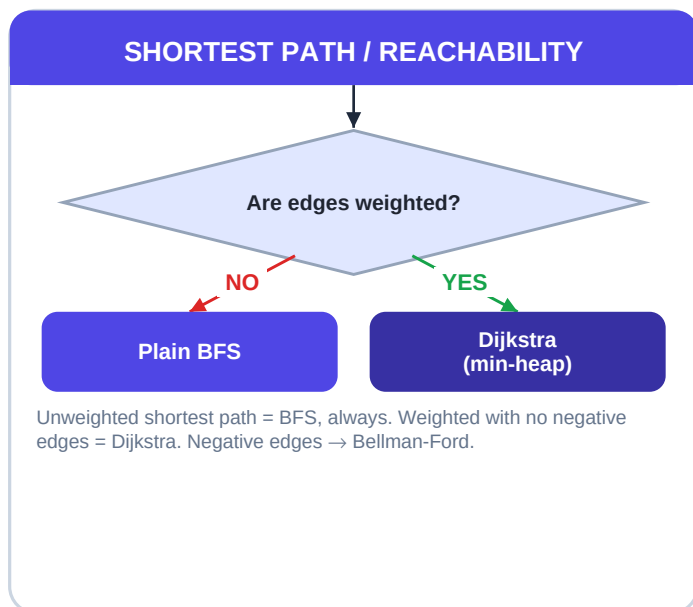
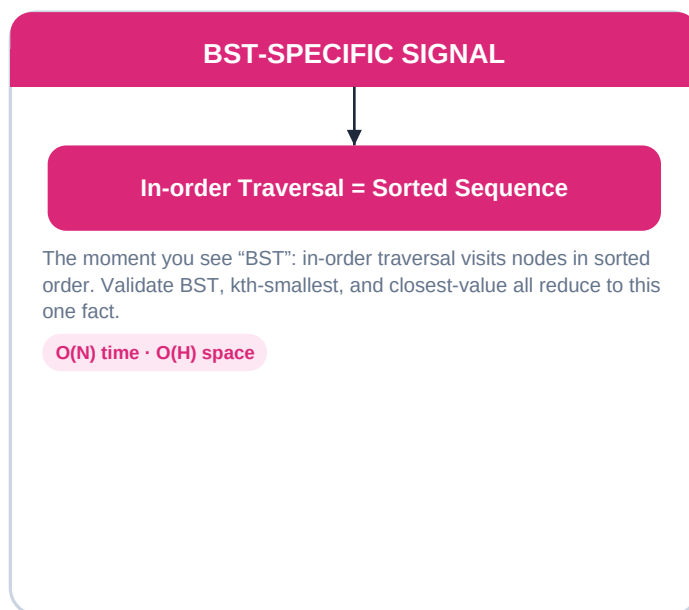
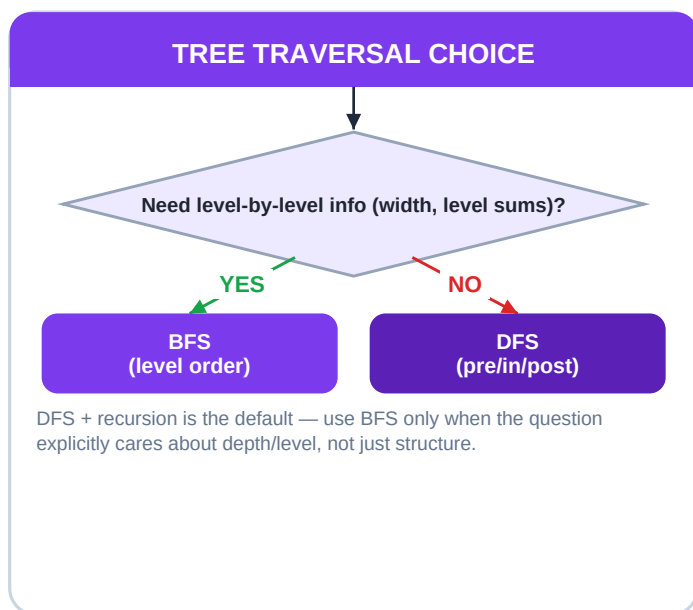
Always draw 4-5 boxes with arrows on paper/whiteboard before touching pointers — linked list bugs are almost always off-by-one node errors, not logic errors.

Ask out loud: "Can the head change?" If yes, use a dummy node. This single question prevents most null-pointer crashes in interviews.

State the invariant per loop iteration (e.g. "slow is always at the midpoint of what fast has covered") — interviewers grade your reasoning, not just working code.

Pattern Flowchart: Trees & Graphs

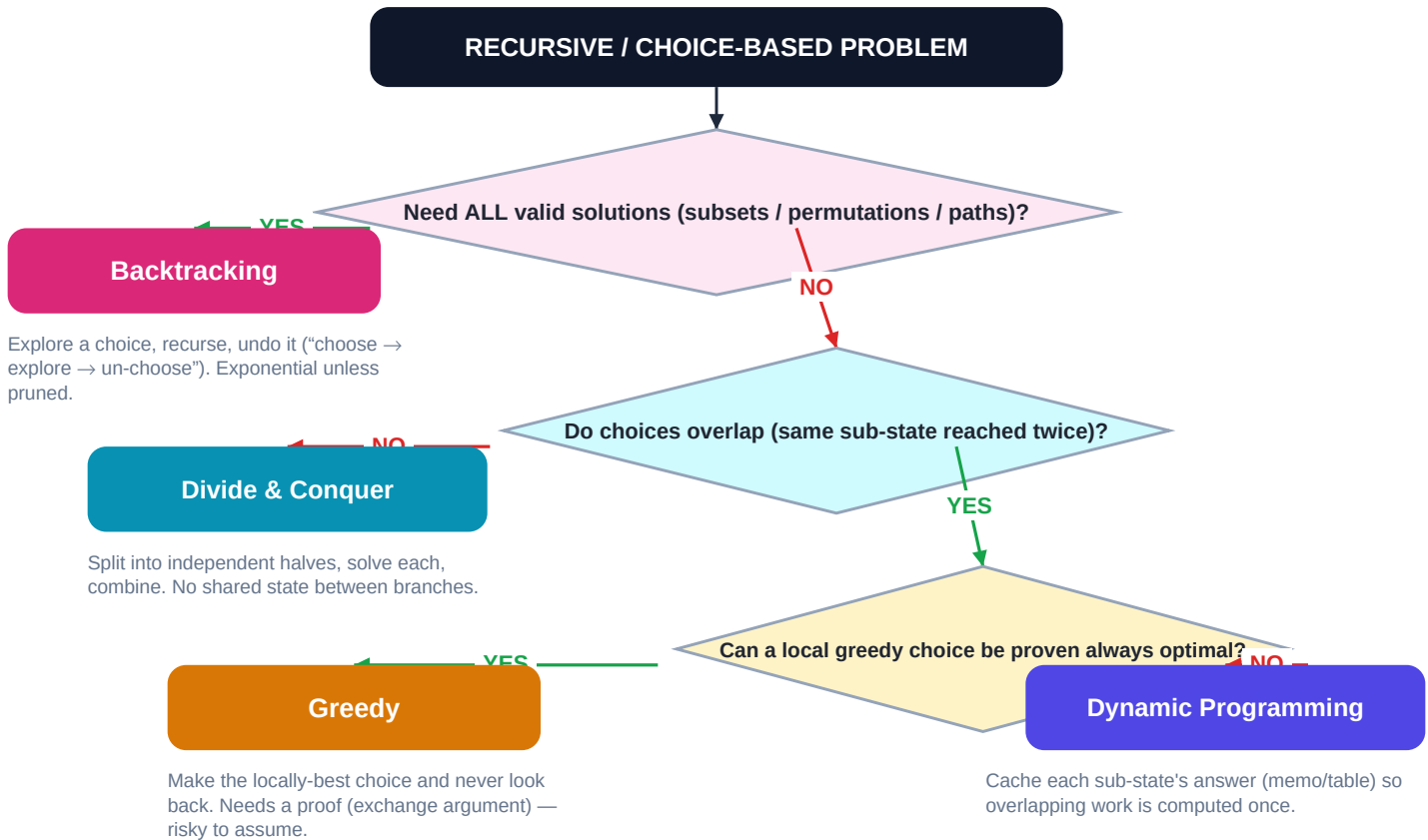
A grid/matrix is just a graph in disguise — apply the same BFS/DFS/Union-Find logic with 4-directional neighbors instead of an adjacency list.



Grid problems: "number of islands", "rotting oranges", "shortest path in binary matrix" are all Graph BFS/DFS wearing a matrix costume — treat each cell as a node with up to 4 neighbors.

Backtracking vs DP vs Greedy vs D&C

The four patterns that make people freeze in interviews — because they all "use recursion." The real question is never "is this recursive?" It's what you do with the recursion.



DP sub-pattern cheat — once you know it's DP, which shape?

0/1 Knapsack

Each item used at most once; pick subset under a capacity.

"each item once", "subset sum", "partition equal sum"

Unbounded Knapsack

Each item reusable unlimited times.

"coin change", "rod cutting", "unlimited supply"

LIS (Longest Increasing Subsequence)

Best chain of elements keeping relative order.

"longest increasing/valid subsequence"

LCS / Edit Distance

Compare two sequences position by position.

"common subsequence", "convert word A to B"

Interval DP

Answer for a range depends on splitting that range.

"matrix chain", "burst balloons", "palindrome partition"

DP on Grid

Path counting/optimizing moving only right/down.

"unique paths", "min path sum"

Constraint → Complexity → Pattern

The single highest-leverage habit in competitive interviews: read the constraint on N before you think about the algorithm. It tells you the expected time complexity — which eliminates most wrong patterns immediately.

INPUT SIZE (N)	TIME BUDGET	PATTERNS THAT FIT	RULED OUT
$N \leq 10-12$	$O(2^N)$ or $O(N!)$	Brute force, Backtracking (all subsets/permutations)	Anything polynomial-only — problem wants exhaustive search
$N \leq 20-24$	$O(2^N \cdot N)$	Bitmask DP (state = subset of items)	Plain backtracking without memoization (too slow)
$N \leq 100-500$	$O(N^3)$	DP with 2D table + $O(N)$ transition, Floyd-Warshall	$O(N^2)$ is probably too simple for what's being asked
$N \leq 1,000-2,000$	$O(N^2)$	Nested loops, simple 2D DP, brute-force pairs	$O(N)$ alone is likely insufficient — relation is pairwise
$N \leq 10^5-10^6$	$O(N \log N)$	Sorting, Binary Search, Heap, Merge Intervals, Divide & Conquer	$O(N^2)$ brute force will time out (TLE)
$N \leq 10^6-10^7$	$O(N)$	Two Pointers, Sliding Window, Prefix Sum, HashMap, single-pass Greedy	Anything with nested loops or repeated sorting
$N \leq 10^9$	$O(\log N)$ or $O(\sqrt{N})$	Binary Search on the answer, math/number theory, matrix exponentiation	Any $O(N)$ pass over the full input — too slow to even iterate once
$N \leq 10^{18}$	$O(1)$ or $O(\log N)$	Closed-form formula, fast exponentiation, digit DP	Anything that touches every value — must reason about N symbolically

How to use this at the whiteboard

The moment you read constraints, say the complexity budget out loud: “ N is up to 10^5 , so I'm aiming for $O(N \log N)$ or better.”

If your first idea is $O(N^2)$ but $N \leq 10^5$, don't code it — say so, then re-derive using the table above.

Time limits are usually tuned for $\sim 10^8$ operations/second. Use that to sanity-check any complexity guess.

Rule of thumb: 10^8 basic operations $\approx$ 1 second. $N \leq 10^5$ with $O(N^2) = 10^{10}$ ops $\rightarrow$ will TLE.

Keyword → Pattern Trigger List

The exact phrasing interviewers use to describe a problem. Recognize these on sight and you'll name the pattern before you've finished reading the prompt.

ARRAY / STRING		GRAPH / TREE / LINKED LIST	
"two sum" / "pair with sum"	HashMap	"number of islands"	BFS/DFS (grid)
"sorted array" + "find target"	Binary Search	"course schedule" / prerequisites	Topological Sort
"rotated sorted array"	Binary Search (mod.)	"clone graph"	BFS/DFS + HashMap
"first/last occurrence"	Binary Search	"shortest path" (unweighted)	BFS
"kth largest/smallest"	Heap	"shortest path" (weighted)	Dijkstra
"top k frequent"	Heap + HashMap	"detect cycle" (linked list)	Fast & Slow
"longest substring w/o repeat"	Sliding Window	"reverse linked list"	Iterative reversal
"minimum window substring"	Sliding Window	"merge two sorted lists"	Dummy node
"max sum subarray, size k"	Sliding Window	"nth node from end"	Two ptr + gap
"subarray sum equals k"	Prefix Sum	BACKTRACKING / DP	
"max subarray sum" (Kadane)	DP / Greedy	"all subsets" / "power set"	Backtracking
"merge intervals"	Sort + Merge	"all permutations"	Backtracking
"meeting rooms"	Sort + Heap	"combination sum"	Backtracking
"next greater element"	Monotonic Stack	"n-queens" / "sudoku solver"	Backtracking
"daily temperatures"	Monotonic Stack	"ways to climb / reach"	DP (1D)
"sliding window maximum"	Monotonic Deque	"0/1 knapsack" / "subset sum"	DP (0/1 knapsack)
"valid parentheses"	Stack	"coin change"	DP (unbounded)
		"longest common subsequence"	DP (2D)
		"edit distance"	DP (2D)
		"unique paths" (grid)	DP on grid
		"word break"	DP + HashSet
		DESIGN / STREAMING	
		"design LRU cache"	HashMap + DLL
		"median of a stream"	Two Heaps
		"running/live top k"	Heap

None of these match exactly? Fall back to the 3-Signal Method on page 2 — input shape, keywords, constraints — and work it out from first principles.

Master Cheat Sheet

Every pattern in this guide, one line each. Read this page the morning of your interview.

Two Pointers Sorted data, pair/triplet search $O(N)$	Sliding Window Contiguous range, fixed/variable size $O(N)$	HashMap / Set $O(1)$ lookup, complement/frequency $O(N)$
Prefix Sum Range sum queries, has negatives $O(N)$	Binary Search Sorted, or monotonic answer space $O(\log N)$	Heap / Priority Queue Kth element, top-k, merge k lists $O(N \log k)$
Sort + Merge Intervals Overlapping ranges, scheduling $O(N \log N)$	Monotonic Stack Next greater/smaller, spans $O(N)$	Fast & Slow Pointers Cycle, middle node, palindrome $O(N)$
Dummy Node Merge/splice linked lists $O(N)$	BFS Shortest path, level order $O(V+E)$	DFS Connectivity, all paths, trees $O(V+E)$
Union-Find Components, cycle in undirected $O(N \alpha(N))$	Topological Sort Dependency/build order $O(V+E)$	Backtracking All subsets/permutations/paths $O(2^N)$ or $O(N!)$
Dynamic Programming Overlapping subproblems, optimal value varies	Greedy Provably-safe local optimal choice $O(N \log N)$	Divide & Conquer Independent halves, combine $O(N \log N)$

How to structure your answer out loud

1. Clarify

Restate constraints, ask about duplicates/negatives/empty input, confirm N's range.

2. Name the pattern

"Given it's sorted, I'll use Two Pointers" — signal before solution, always.

3. Walk the approach

Describe the algorithm in plain English before writing a single line of code.

4. Code

Write clean code; narrate as you go so silence never fills the room.

5. Test

Trace through one normal case and one edge case (empty, single element, all same).

6. Complexity

State time and space complexity unprompted — don't wait to be asked.

Good luck, Wasim. Target: Senior Software Engineer / Tech Lead, Gulf market, 25,000+ AED/SAR. You've got the framework — now put in the reps.